Lab 7: Fuzzing for Input Validation Bugs

Module: **bike_share_rental_parser.py** (parse_ride_duration, format_station_code, parse_ride_log)

Student	Hemanth (GitHub: SHemanth07)
SRN	PES1UG24CS418
Tools	Python 3.14, pytest 9.1.1, Hypothesis 6.170.15
GitHub repo	github.com/SHemanth07/SE-Labs-PES1UG24CS418 (folder: lab7)

Summary

The original module assumes every input is well formed. Hypothesis fuzz tests found that all three functions crash on empty strings, `None`, wrong types and malformed text. Each function now validates its input and returns `None` for invalid input instead of raising, and `main()` prints a clear message. After the fix all 9 tests pass (6 failed on the buggy module).

Bugs found and how they were fixed

Function	Failing input (found by Hypothesis)	Original crash	Fix
parse_ride_duration	None	AttributeError	Reject non-strings
	'', '75', '1:2:3'	ValueError (unpack)	Require exactly 2 parts split by ':'
	':', 'ab:cd'	ValueError (int)	ASCII-digit check; minutes 0-59
format_station_code	'' (empty)	IndexError	Require length >= 2
	None, b'\x00' (bytes)	AttributeError / TypeError	Reject non-strings; letter + digits only
parse_ride_log	'', 'a,5'	IndexError	Require exactly 3 fields
	',', 'a,b,c'	ValueError (int)	Validate minutes as a positive integer
	None, b''	AttributeError / TypeError	Reject non-strings; subscriber must be yes/no

SS1: Original (buggy) module

```
sanap@HemanthKumar07 MINGW64 ~/SE-Labs-PES1UG24CS418/lab7 (main)
$ python bike_share_rental_parser.py
==== Running parse_ride_duration ====
Enter a ride duration as HH:MM (e.g. 01:15): 01:15
Duration in minutes: 75

==== Running format_station_code ====
Enter a station code (e.g. a12): a12
Formatted station code: A-12

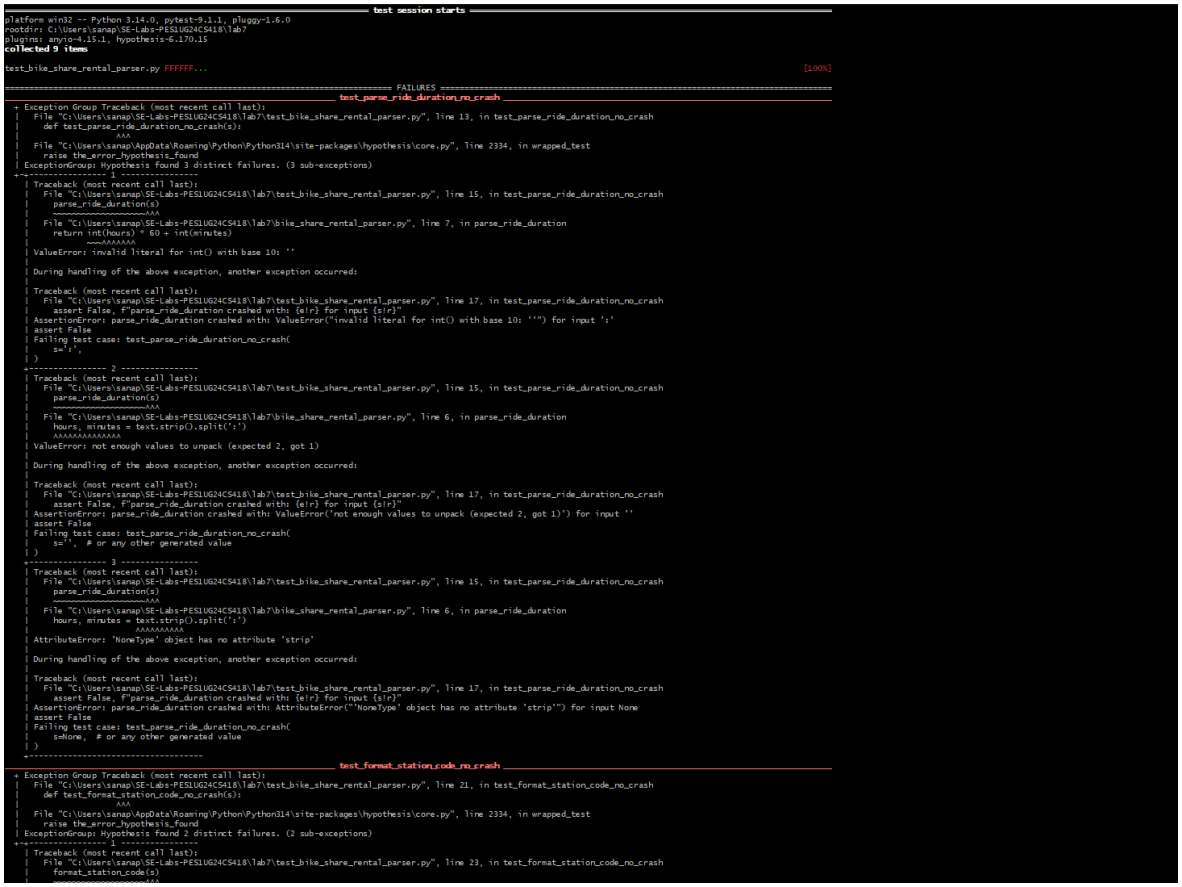
==== Running parse_ride_log ====
Enter a ride log line as type,minutes,subscriber (e.g. electric,45,yes): electric,45,yes
Parsed ride log: ('electric', 45, True)

sanap@HemanthKumar07 MINGW64 ~/SE-Labs-PES1UG24CS418/lab7 (main)
$ python bike_share_rental_parser.py
==== Running parse_ride_duration ====
Enter a ride duration as HH:MM (e.g. 01:15): 75
Traceback (most recent call last):
  File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\bike_share_rental_parser.py", line 39, in <module>
    main()
    ~~~~^^
  File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\bike_share_rental_parser.py", line 28, in main
    print("Duration in minutes:", parse_ride_duration(value))
                                  ~~~~~~~~~~~~~~~~~~~^^^^^^^
  File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\bike_share_rental_parser.py", line 6, in parse_ride_duration
    hours, minutes = text.strip().split(':')
    ^^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
```

SS1: python bike_share_rental_parser.py on the buggy module. Valid input works; input '75' crashes with ValueError.

SS2: Fuzz tests failing on the buggy module

pytest test_bike_share_rental_parser.py: 9 items collected, 6 failed, 3 passed (parts 1-5 below).

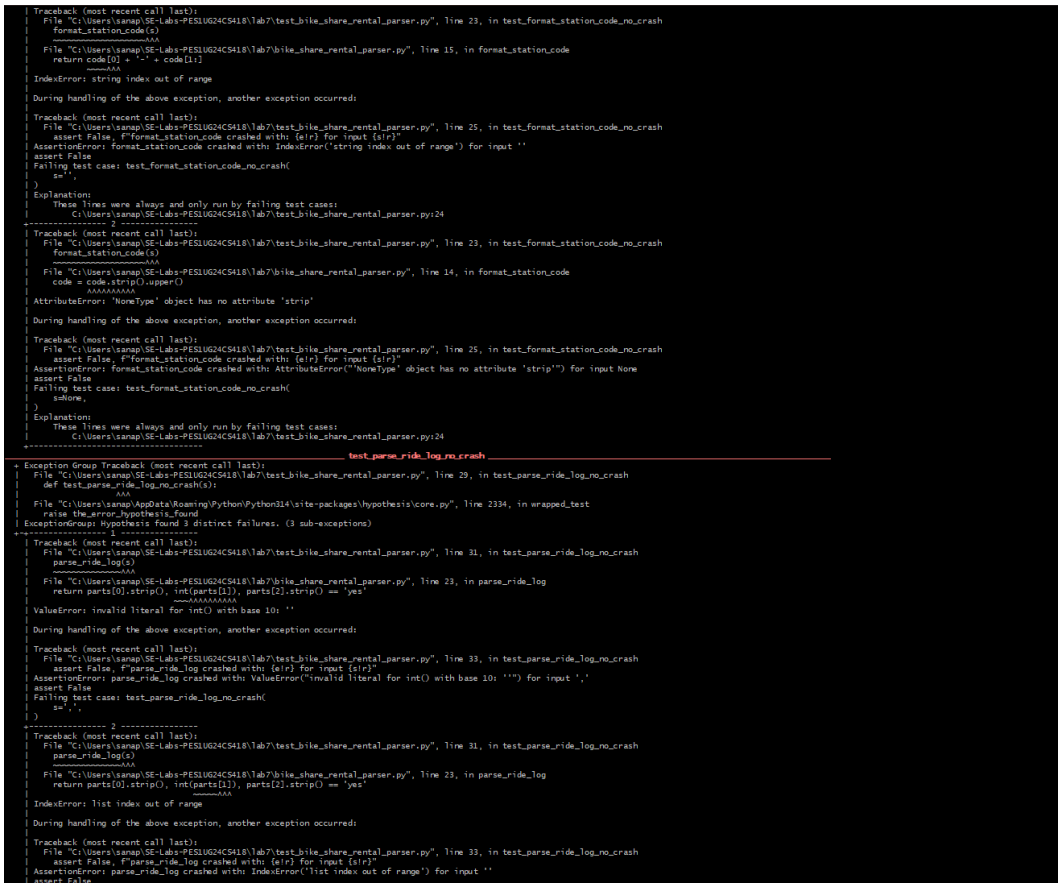


```
test session starts
platform win32 -- Python 3.14.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7
plugins: anyio-4.15.1, hypothesis-6.170.15
collected 9 items

test_bike_share_rental_parser.py FFFFFF...                                [100%]

================================== FAILURES ==================================
_____ test_parse_ride_duration_no_crash _____
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\test_bike_share_rental_parser.py", line 13, in test_parse_ride_duration_no_crash
  |     def test_parse_ride_duration_no_crash(s):
  |                ^^^
  |   File "C:\Users\sanap\AppData\Roaming\Python\Python314\site-packages\hypothesis\core.py", line 2334, in wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\test_bike_share_rental_parser.py", line 15, in test_parse_ride_duration_no_crash
    |     parse_ride_duration(s)
    |     ~~~~~~~~~~~~~~~~~~~^^^
    |   File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\bike_share_rental_parser.py", line 7, in parse_ride_duration
    |     return int(hours) * 60 + int(minutes)
    |                ~~~^^^^^^^
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\test_bike_share_rental_parser.py", line 17, in test_parse_ride_duration_no_crash
    |     assert False, f"parse_ride_duration crashed with: {e!r} for input {s!r}"
    | AssertionError: parse_ride_duration crashed with: ValueError("invalid literal for int() with base 10: ''") for input ':'
    | assert False
    | Falsing test case: test_parse_ride_duration_no_crash(
    |     s=':',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\test_bike_share_rental_parser.py", line 15, in test_parse_ride_duration_no_crash
    |     parse_ride_duration(s)
    |     ~~~~~~~~~~~~~~~~~~~^^^
    |   File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\bike_share_rental_parser.py", line 6, in parse_ride_duration
    |     hours, minutes = text.strip().split(':')
    |     ^^^^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\test_bike_share_rental_parser.py", line 17, in test_parse_ride_duration_no_crash
    |     assert False, f"parse_ride_duration crashed with: {e!r} for input {s!r}"
    | AssertionError: parse_ride_duration crashed with: ValueError('not enough values to unpack (expected 2, got 1)') for input ''
    | assert False
    | Falsing test case: test_parse_ride_duration_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\test_bike_share_rental_parser.py", line 15, in test_parse_ride_duration_no_crash
    |     parse_ride_duration(s)
    |     ~~~~~~~~~~~~~~~~~~~^^^
    |   File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\bike_share_rental_parser.py", line 6, in parse_ride_duration
    |     hours, minutes = text.strip().split(':')
    |                      ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'strip'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\test_bike_share_rental_parser.py", line 17, in test_parse_ride_duration_no_crash
    |     assert False, f"parse_ride_duration crashed with: {e!r} for input {s!r}"
    | AssertionError: parse_ride_duration crashed with: AttributeError("'NoneType' object has no attribute 'strip'") for input None
    | assert False
    | Falsing test case: test_parse_ride_duration_no_crash(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
_____ test_format_station_code_no_crash _____
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\test_bike_share_rental_parser.py", line 21, in test_format_station_code_no_crash
  |     def test_format_station_code_no_crash(s):
  |                ^^^
  |   File "C:\Users\sanap\AppData\Roaming\Python\Python314\site-packages\hypothesis\core.py", line 2334, in wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7\test_bike_share_rental_parser.py", line 23, in test_format_station_code_no_crash
    |     format_station_code(s)
    |     ~~~~~~~~~~~~~~~~~~~^^^
```

SS2 (part 1 of 5)



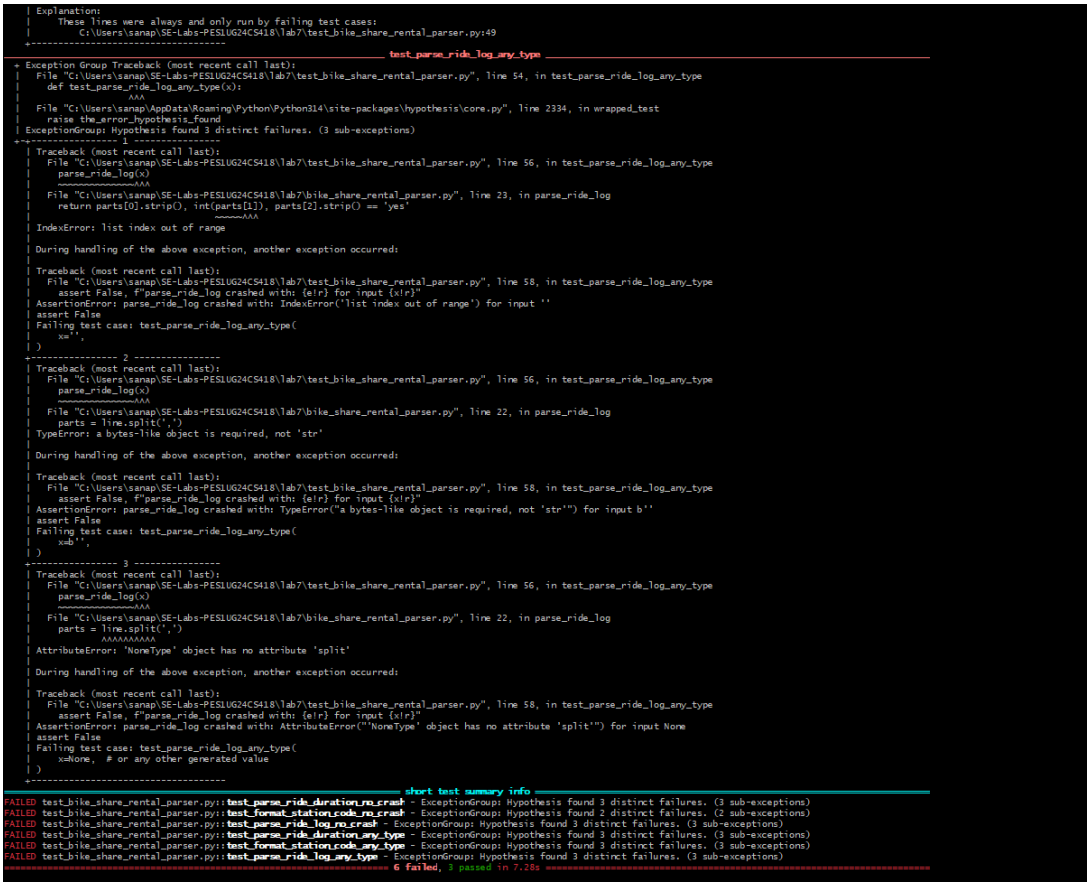
SS2 (part 2 of 5)



SS2 (part 3 of 5)



SS2 (part 4 of 5)



SS2 (part 5 of 5)

SS3: Corrected module

```
sanap@HemanthKumar07 MINGW64 ~/SE-Labs-PES1UG24CS418/lab7 (main)
$ python bike_share_rental_parser.py
==== Running parse_ride_duration ====
Enter a ride duration as HH:MM (e.g. 01:15): 01:15
Duration in minutes: 75

==== Running format_station_code ====
Enter a station code (e.g. a12): a12
Formatted station code: A-12

==== Running parse_ride_log ====
Enter a ride log line as type,minutes,subscriber (e.g. electric,45,yes): electric,45,yes
Parsed ride log: ('electric', 45, True)

sanap@HemanthKumar07 MINGW64 ~/SE-Labs-PES1UG24CS418/lab7 (main)
$ python bike_share_rental_parser.py
==== Running parse_ride_duration ====
Enter a ride duration as HH:MM (e.g. 01:15): abc
Invalid duration: 'abc'

==== Running format_station_code ====
Enter a station code (e.g. a12): x
Invalid station code: 'x'

==== Running parse_ride_log ====
Enter a ride log line as type,minutes,subscriber (e.g. electric,45,yes): y
Invalid ride log line: 'y'
```

SS3: python bike_share_rental_parser.py on the fixed module. Valid input works; invalid input prints 'Invalid ...' messages instead of crashing.

SS4: Final test run (all passing)

```
sanap@HemanthKumar07 MINGW64 ~/SE-Labs-PES1UG24CS418/lab7 (main)
$ python -m pytest test_bike_share_rental_parser.py
============================= test session starts =============================
platform win32 -- Python 3.14.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\sanap\SE-Labs-PES1UG24CS418\lab7
plugins: anyio-4.15.1, hypothesis-6.170.15
collected 9 items

test_bike_share_rental_parser.py .........                               [100%]

============================== 9 passed in 1.91s ==============================
```

SS4: pytest test_bike_share_rental_parser.py on the fixed module: 9 passed.

Source code: bike_share_rental_parser.py (fixed)

```
"""
Bike share rental parser (fixed version).

Every function validates its input and returns None when the input is
invalid, instead of crashing with an exception. Valid input behaves
exactly as before.
"""


def _is_ascii_digits(s):
    """True only for a non-empty string made of ASCII digits 0-9."""
    return isinstance(s, str) and s != "" and s.isascii() and s.isdigit()


def parse_ride_duration(text):
    """
    Converts a ride duration written as HH:MM into minutes.
    Returns the number of minutes, or None if the input is invalid.
    """
    if not isinstance(text, str):
        return None
    parts = text.strip().split(':')
    if len(parts) != 2:
        return None
    hours, minutes = parts
    if not (_is_ascii_digits(hours) and _is_ascii_digits(minutes)):
        return None
    if len(hours) > 4 or len(minutes) > 2:
        return None
    hours, minutes = int(hours), int(minutes)
    if minutes > 59:
        return None
    return hours * 60 + minutes


def format_station_code(code):
    """
    Formats a station code as ZONE-NUMBER, e.g. 'a12' -> 'A-12'.
    Returns None if the code is not one letter followed by digits.
    """
    if not isinstance(code, str):
        return None
    code = code.strip()
    if len(code) < 2 or not code.isascii():
        return None
    if not code[0].isalpha() or not _is_ascii_digits(code[1:]):
        return None
    code = code.upper()
    return code[0] + '-' + code[1:]


def parse_ride_log(line):
    """
    Parses 'bike_type,minutes,subscriber' into (str, int, bool).
    Returns None if the line does not have exactly three valid fields.
    """
    if not isinstance(line, str):
        return None
    parts = line.split(',')
    if len(parts) != 3:
        return None
    bike_type = parts[0].strip()
    minutes = parts[1].strip()
    subscriber = parts[2].strip().lower()
    if bike_type == "":
        return None
    if not _is_ascii_digits(minutes) or len(minutes) > 6 or int(minutes) <= 0:
        return None
    if subscriber not in ("yes", "no"):
        return None
    return bike_type, int(minutes), subscriber == "yes"


def main():
    try:
        print("==== Running parse_ride_duration ====")
```

```
        value = input("Enter a ride duration as HH:MM (e.g. 01:15): ")
        result = parse_ride_duration(value)
        print("Duration in minutes:" if result is not None else "Invalid duration:",
              result if result is not None else repr(value))

        print("\n==== Running format_station_code ====")
        value = input("Enter a station code (e.g. a12): ")
        result = format_station_code(value)
        print("Formatted station code:" if result is not None else "Invalid station code:",
              result if result is not None else repr(value))

        print("\n==== Running parse_ride_log ====")
        value = input("Enter a ride log line as type,minutes,subscriber (e.g. electric,45,yes): ")
        result = parse_ride_log(value)
        print("Parsed ride log:" if result is not None else "Invalid ride log line:",
              result if result is not None else repr(value))
    except (EOFError, KeyboardInterrupt):
        print("\nInput cancelled.")


if __name__ == "__main__":
    main()
```

Source code: test_bike_share_rental_parser.py (Hypothesis tests)

```
from hypothesis import given, settings, strategies as st
from bike_share_rental_parser import parse_ride_duration, format_station_code, parse_ride_log

# Anything a careless caller might pass in, not just strings.
ANYTHING = st.one_of(
    st.none(), st.booleans(), st.integers(), st.floats(),
    st.binary(), st.lists(st.integers()), st.text(),
)


# ---------- 1. Fuzz tests: no function may crash on any input ----------
@given(st.text() | st.none())
def test_parse_ride_duration_no_crash(s):
    try:
        parse_ride_duration(s)
    except Exception as e:
        assert False, f"parse_ride_duration crashed with: {e!r} for input {s!r}"


@given(st.text() | st.none())
def test_format_station_code_no_crash(s):
    try:
        format_station_code(s)
    except Exception as e:
        assert False, f"format_station_code crashed with: {e!r} for input {s!r}"


@given(st.text() | st.none())
def test_parse_ride_log_no_crash(s):
    try:
        parse_ride_log(s)
    except Exception as e:
        assert False, f"parse_ride_log crashed with: {e!r} for input {s!r}"


# ---------- 2. Fuzz tests with non-string inputs ----------
@given(ANYTHING)
def test_parse_ride_duration_any_type(x):
    try:
        parse_ride_duration(x)
    except Exception as e:
        assert False, f"parse_ride_duration crashed with: {e!r} for input {x!r}"


@given(ANYTHING)
def test_format_station_code_any_type(x):
    try:
        format_station_code(x)
    except Exception as e:
        assert False, f"format_station_code crashed with: {e!r} for input {x!r}"


@given(ANYTHING)
def test_parse_ride_log_any_type(x):
    try:
        parse_ride_log(x)
    except Exception as e:
        assert False, f"parse_ride_log crashed with: {e!r} for input {x!r}"


# ---------- 3. Property tests: valid input still gives the right answer ----------
@given(st.integers(0, 99), st.integers(0, 59))
def test_valid_duration_round_trip(h, m):
    assert parse_ride_duration(f"{h:02d}:{m:02d}") == h * 60 + m


@given(st.sampled_from("abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"),
       st.integers(0, 9999))
def test_valid_station_code_format(letter, number):
    assert format_station_code(f"{letter}{number}") == f"{letter.upper()}-{number}"


@given(st.sampled_from(["standard", "electric", "cargo"]),
       st.integers(1, 100000), st.booleans())
def test_valid_ride_log(bike, minutes, sub):
    line = f"{bike},{minutes},{'yes' if sub else 'no'}"
```

```
    assert parse_ride_log(line) == (bike, minutes, sub)
```